```
  }
}
```

❶ Nesting is done by grouping with {}.

For simple applications, this file often suffices. The format looks somewhat like JSON. The primary advantages are that it's more readable and it's easy to see how properties are grouped. JDBC is a perfect example of properties that most apps need and that are easier to manage when grouped. In the dependency injection world, you would group items like this by controlling the injection of the properties into objects (such as `DataSource`). This is a simpler approach. Let's look at how you can use these properties now that you have an initial understanding of them.

You can use several methods to get the values as different types, and the period (`.`) is used as the separator in the path of the property. The basic types are supported, and you can get lists of these types:

```
val applicationVersion = config.getInt("MyAppl.version")                ❶
val databaseConnectString = config.getString("MyAppl.database.connect") ❷
```

❶ Gets the "MyApp1.version" value from application.conf

❷ Gets the "MyAppl.database.connect" value from application.conf

7.1.2 Subtrees

Sometimes an object doesn't need much configuration. What if you have a class `DBaseConnection` that's creating the database connection? This class needs only the connect string and the `user` property. When you pass the full configuration to the `DBaseConnection` class, it needs to know the full path of the property. But when you want to reuse `DBaseConnection` in